



UNIVERSITY OF DHAKA

**Design and Implementation of a
Scalable and Secure Single
Sign-On (SSO) Framework with
Cross-Domain Identity Federation
in Multi-Tenant Environments**

Submitted by

Class Roll:40007

Registration No:H-121

Class Roll:40026

Registration No:H-129

Submitted to the

Department of Computer Science and Engineering

University of Dhaka, Bangladesh

**in partial fulfillment of the requirements for the degree of
Professional Masters in Information and Cyber Security (PMICS).**



UNIVERSITY OF DHAKA

Design and Implementation of a Scalable and Secure Single Sign-On (SSO) Framework with Cross-Domain Identity Federation in Multi-Tenant Environments

Submitted by

Class Roll:40007

Registration No:H-121

Class Roll:40026

Registration No:H-129

Submitted to the

Department of Computer Science and Engineering

University of Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of
Professional Masters in Information and Cyber Security (PMICS).

Abstract

Organizations are progressively utilizing multi-tenant architectures and federated services in cloud environments to enhance resource utilization and collaboration across domains. However, secure and efficient user authentication and authorization management continue to be a significant challenge. This project suggests the development and execution of a secure and scalable Single-Sign-On (SSO) framework that facilitates cross-domain identity federation in multi-tenant environments. The framework ensures tenant isolation, security, and performance while facilitating seamless user access across organizational boundaries by utilizing open standards such as OAuth 2.0 and OpenID Connect. The system addresses critical challenges such as cross-domain policy enforcement, scalability, and authentication interoperability by integrating federated identity providers (IdPs), role-mapping mechanisms, and fine-grained access control. The prototype that has been created illustrates secure Single-Sign-On (SSO) authentication, dynamic tenant on-boarding, and real-time user federation with minimal latency. This work is a practical and viable solution for enterprise-grade cloud systems equipped with federated service ecosystems, contributing to the advancement of secure identity management architectures.

Contents

Declaration Form	i
Abstract	ii
Acknowledgements	iii
List of Figures	vii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	5
1.2.1 Lack of Federation Support	5
1.2.2 Inadequate Tenant Isolation	5
1.2.3 Rigid Architectures	5
1.2.4 Persistent Credential Overload	6
1.2.5 Limited Scalability	6
1.3 Objectives of the Project	7
1.4 Expected Outcomes	8
1.4.1 Prototype Implementation and Demonstration	8
1.4.2 Enhanced User Experience and Accessibility	8
1.4.3 Strengthened Security Posture	8
1.4.4 Deployment-Ready and Extensible Architecture	9
1.4.5 Practical and Industry-Oriented Deliverables	9
1.5 Contributions	10
1.5.1 Contributions to the Academic Community	10
1.5.2 Significant Real-World Effects	10
2 Related Works	11
2.1 Evolution of SSO: From Single-Domain to Cross-Domain Trust . . .	11
2.2 Federation Protocols and Interoperability	13
2.3 Multi-Tenancy Patterns in IAM	14
2.4 Access Control: RBAC, ABAC, and Policy Federation	15
2.5 Security Hardening, Observability, and Logout Semantics	16
2.6 Passwordless and Zero-Trust Authentication	17
2.7 Representative Studies and Research Gaps	18

3	Research and Methodology	19
3.1	Research Approach	19
3.1.1	Problem Identification and Motivation	20
3.1.2	Requirement Definition	20
3.1.2.1	Multi-Protocol Support	20
3.1.2.2	Federated Trust Establishment	21
3.1.2.3	Tenant-Aware Realm Isolation	22
3.1.2.4	Passwordless Readiness	22
3.1.2.5	Observability and Auditability	23
3.1.3	Design and Development of the Framework	24
3.1.3.1	Hub-and-Spoke Architecture	24
3.1.3.2	Federation Broker (Keycloak)	24
3.1.3.3	Persistent Storage (PostgreSQL)	25
3.1.3.4	Reverse Proxy & Load Balancer (NGINX)	25
3.1.3.5	Service Provider (SP) Integration	25
3.1.3.6	Implementation and Prototyping	27
3.1.3.7	Evaluation and Validation	29
3.1.3.8	Communication of Results	29
3.2	Research Design	31
3.3	Methodology	33
3.3.1	Step-by-Step Implementation	33
3.3.2	Tool-by-Tool Rationale	34
3.3.3	Authentication and Logout Flow	35
3.3.4	Use Cases	37
3.4	Evaluation Framework	38
3.4.1	Metrics and Instruments	38
3.4.2	Experimental Design	38
3.5	Methodological Justification and Contributions	39
4	Results and Development	40
4.1	Implementation of the SSO Framework	40
4.1.1	4.1.1 Environment Setup	41
4.1.2	Database Setup	41
4.1.3	Keycloak Installation and Configuration	43
4.1.4	Configure Access Control	43
4.1.5	Service Provider Integration	44
4.1.6	Reverse Proxy and Load Balancer Setup	46
4.1.7	Implement Security Features	46
4.2	Functional Testing of the Single Sign-On (SSO) Framework	47
4.2.1	Key Functional Testing Areas	47
4.2.2	Functional Testing Results	49
4.3	Feature Testing	49
4.3.1	Key Features Tested:	49
4.3.2	Feature Testing Results:	49

4.4	Results from Deployed Project	51
4.4.1	Deployed System Overview	51
4.4.2	Deployed System Testing Results	51
4.5	Shortcomings	53
4.6	Summary	54
5	Conclusions	55
5.1	Summary of Work	55
5.2	Multi-Protocol Federation & Interoperability	56
5.3	Hub-and-Spoke Architecture for Centralized Control	57
5.4	Isolation and Fine-Grained Access Control for Tenants	58
5.5	Improved Security via Passwordless Authentication	59
5.6	Security Hardening and Real-Time Monitoring	60
5.7	Scalability and Performance	61
5.8	Real-World Application and Integration	62
5.9	Compliance and Observability	64
5.10	Practical Contributions and Impact	65
	Bibliography	66

List of Figures

3.1	Hub and Spoke with Federation Broker (Keycloak)	27
3.2	Hub-and-Spoke Architecture	32
3.3	SSO Authentication Process	36
3.4	End-to-End Authentication & Single Logout Flow	36
4.1	Users list of SMS Apps	42
4.2	Integration with Google	44
4.3	Showing database from KEYCLOCK	45
4.4	Showing database from CLI	45
4.5	Showing database from Apps Interface	45

Chapter 1

Introduction

1.1 Background and Motivation

The modern enterprise computing landscape is undergoing a profound transformation, driven by the accelerated adoption of multi-cloud deployments, hybrid infrastructures, and cross-organizational collaborations.[17, 20] Enterprises no longer confine themselves to isolated, monolithic IT environments. Instead, they routinely operate across multiple domains, integrate numerous third-party services, and engage in partnerships that demand secure, reliable, and efficient identity management across a broad spectrum of platforms. In this context, Identity and Access Management (IAM) has evolved from a purely operational concern into a strategic pillar that underpins business agility, regulatory compliance, and cyber-security resilience.[9, 16]

User authentication has become a persistent challenge in these distributed and heterogeneous environments. Employees, contractors, and partners often require access to a variety of applications, from internal enterprise systems to Software-as-a-Service (SaaS) offerings hosted in the cloud. In many organizations, each application or platform maintains its own authentication system, requiring users to remember multiple usernames and passwords.

This phenomenon, widely known as credential sprawl, has several negative consequences:[19]

- Repetitive login processes across different systems result in reduced productivity.
- Risky behaviors such as password reuse, weak password creation, or reliance on insecure password storage methods lead to a weakened security posture.
- Attackers can easily exploit gaps in authentication enforcement due to inconsistent access policies.

The cumulative effect of these issues create a bigger target for cyberattacks, make users more vulnerable to phishing and stolen-password attacks, and put extra strain on IT teams who have to manage multiple, disconnected identity systems.[10, 18]

To address these challenges, Single-Sign-On (SSO) technologies were developed, allowing users to authenticate once and gain access to multiple applications [15] without repeated credential prompts. By centralizing the authentication process, SSO reduces friction for users and enables organizations to enforce consistent security policies. However, the traditional SSO model is predominantly designed for single-domain environments [7], such as an organization's internal network. In such cases, a single Identity Provider (IdP) handles authentication for all integrated applications, but trust boundaries remain limited to that single organizational domain.

In today's interconnected world, such a narrow scope is insufficient. Modern organizations often require secure collaboration with external entities, including partner firms, cloud service providers, and geographically distributed subsidiaries. Trust boundaries now span corporate, geographic, and technological barriers, making it essential for SSO solutions to extend beyond a single security domain.

This is where identity federation plays a critical role. Identity federation is the process of linking disparate authentication systems so that users from one domain can securely access resources in another, without the need for duplicate accounts. Federation leverages open, standardized protocols—notably Security Assertion Markup

Language (SAML 2.0), OAuth 2.0, and OpenID Connect (OIDC)—to [4, 9, 20] facilitate secure, protocol-compliant authentication between independent IdPs and Service Providers (SPs).

Through federation, organizations can:

- **Maintain full administrative control** over their user directories.
- **Delegate trust** to other IdPs without compromising security.
- Participate in **shared authentication ecosystems** that enable secure, seamless access across organizational boundaries.

However, integrating identity federation into a **multi-tenant SSO framework** introduces a unique set of challenges:

1. Strict Tenant Isolation

In multi-tenant environments, each tenant (organization, department, or client) must have its authentication policies, user data, and access privileges strictly segregated from others to prevent cross-tenant data leakage or unauthorized access.[8, 13]

2. Complex Role and Attribute Mapping

Users authenticated through an external IdP may have roles and attributes defined differently than in the local system. Effective federation requires a reliable mapping mechanism to ensure consistent access control enforcement.[1]

3. Scalable Authentication Architecture

The SSO system must be capable of handling large numbers of tenants and high volumes of authentication requests without performance degradation.

4. Security Assurance Across Domains

Trust relationships between IdPs must be carefully managed and monitored to prevent exploitation. This includes ensuring protocol compliance, certificate management, and continuous verification of configurations.[2, 3]

These challenges are **amplified** by the **business-critical nature** of authentication. A misconfiguration, performance bottleneck, or security breach in the SSO infrastructure can have far-reaching implications, including downtime, regulatory violations, and reputational damage.

These challenges are **amplified** by the **business-critical nature** of authentication. A misconfiguration, performance bottleneck, or security breach in the SSO infrastructure can have far-reaching implications, including downtime, regulatory violations, and reputational damage.

The **motivation** behind this project is therefore twofold:

- The project's technical goal is to design a next-generation SSO implementation that integrates federated identity and multi-tenancy into a scalable, secure, and flexible architecture.
- From a strategic standpoint, the project aims to enhance the existing knowledge in IAM by creating a reference implementation that showcases optimal techniques in federated authentication, tenant-aware access control, and advanced security integration.

The proposed framework is built around **Keycloak**—an open-source, standards-compliant IdP—serving as the **central authentication authority**. *Keycloak's extensibility allows integration with multiple service providers (SPs) and external identity providers (IdPs)* while supporting **Role-Based Access Control (RBAC)**, **Attribute-Based Access Control (ABAC)**, and **passwordless authentication**. Furthermore, the system is designed with **high availability** and **security monitoring** in mind, incorporating TLS encryption, brute-force attack detection, and SIEM integration for real-time incident response.

By implementing SSO with multitenancy using this framework, the project aims to deliver a **practical and deployable solution** to one of today's most critical IAM challenges: **providing seamless, secure, and scalable authentication** in a federated, multi-tenant environment.

1.2 Problem Statement

The present single sign-on (SSO) solutions, notwithstanding the development of identity and access management (IAM) technologies, have a number of significant drawbacks when applied to federated, multi-tenant systems.

1.2.1 Lack of Federation Support

The native support for seamless integration with external identity providers (IdPs) like Azure Active Directory, Google Workspace, or traditional SAML-based providers is lacking in many systems. Environments that support federation often face configuration challenges, fragility, and scope restrictions.[5, 9, 20]

1.2.2 Inadequate Tenant Isolation

It is necessary to maintain a rigorous separation of authentication procedures, user data, and access privileges in settings that contain several tenants. Inadequate isolation techniques can lead to privilege escalation, data leakage, and cross-tenant policy violations.[8, 14]

1.2.3 Rigid Architectures

It is difficult to establish granular access control, policy-driven authentication, or compliance-specific procedures when using monolithic identity and access management (IAM) solutions since these solutions are frequently dependent on a single vendor and are resistant to customization.[7]

1.2.4 Persistent Credential Overload

As a result of the lack of passwordless authentication, users continue to manage several credentials, which makes them more vulnerable to attacks such as phishing, credential stuffing, and social engineering.[4, 10]

1.2.5 Limited Scalability

A significant number of single sign-on (SSO) systems are not built to be horizontally scalable, do not integrate with Security Information and Event Management (SIEM) solutions, and have difficulty accommodating dynamic onboarding of new tenants and identity providers at the same time.

Because of these restrictions, there is an increased demand for a single sign-on (SSO) framework that is both flexible and complies with standards. This framework should be able to function across several identity systems, ensure that tenants are kept separate from one another, and enable various authentication and access control methods.[1]

1.3 Objectives of the Project

The overarching goal of this project is to create, deploy, and assess an SSO framework that is both secure and scalable enough to accommodate cross-domain identity federation in settings with several tenants. A streamlined and integrated authentication process will be made possible by the system's ability to integrate with various internal and external identity providers.[5]

The project aims to achieve the following specific objectives: Connect different identity providers (IdPs) safely and following industry rules, making it simple to log in across different platforms using services like Google Workspace, Azure Active Directory, and custom SAML/OIDC providers.

- **Federated Identity Integration** Implement secure, standards-compliant integration with multiple IdPs (Azure AD, Google Workspace, and custom SAML/OIDC providers), ensuring seamless cross-domain authentication.[9]
- **Tenant-Aware Authentication and Authorization** Enforce strict tenant boundaries, ensuring that authentication policies, credentials, and roles are isolated while still enabling centralized governance.[8]
- **Hybrid Access Control Models** Incorporate both RBAC and ABAC to allow dynamic permission assignment based on user roles, contextual attributes, and tenant-specific policies.[8]
- **Scalable, Modular Architecture** Design a cloud-ready, horizontally scalable system capable of integrating with monitoring and threat detection tools while supporting high availability.[3]

1.4 Expected Outcomes

All of the following outcomes are anticipated to be delivered by the project once it has been successfully completed:

1.4.1 Prototype Implementation and Demonstration

A fully functional and tested Single Sign-On (SSO) prototype that seamlessly integrates multiple identity providers (IdPs) and service providers (SPs).[15] The system will exhibit secure, standards-compliant identity federation across multi-tenant architectures, supporting real-world enterprise scenarios and cross-domain authentication.

1.4.2 Enhanced User Experience and Accessibility

An optimized and consistent authentication experience across diverse applications and platforms. This includes simplified login procedures, reduced credential fatigue through passwordless options, and uniform authentication workflows that improve user satisfaction, productivity, and accessibility for all user groups.[10]

1.4.3 Strengthened Security Posture

Implementation of robust access control mechanisms, tenant isolation, and adaptive authentication strategies. Security enhancements will include reduced attack surfaces, stricter policy enforcement, improved resilience against credential-based attacks, and comprehensive incident response capabilities through integrated monitoring and logging.[9]

1.4.4 Deployment-Ready and Extensible Architecture

A modular, scalable, and cloud-ready architecture capable of adapting to various organizational requirements. The design will include multi-tenancy support, federation with diverse IdPs, and extensibility to accommodate future IAM advancements, regulatory requirements, and technological shifts.[5]

1.4.5 Practical and Industry-Oriented Deliverables

Clear integration guidelines, deployment manuals, and configuration documentation to facilitate adoption by enterprises. The framework will be supported with tenant onboarding guides and security hardening checklists to ensure smooth implementation in production environments.[3]

1.5 Contributions

1.5.1 Contributions to the Academic Community

1. Put forward a single sign-on (SSO) design for multiple tenants that incorporates cross-domain identity federation while keeping tenants completely isolated.
2. Offer empirical evidence regarding performance and security trade-offs by comparing SAML and OIDC, two federation protocols, in multi-tenant environments. [9, 20]
3. Contribute to the body of knowledge in identity and access management (IAM) by outlining a methodology for assessing federated SSO deployments' security and user experience.
4. Fill a need in the literature by providing a deployment methodology for federated single sign-on systems that use passwordless authentication.

1.5.2 Significant Real-World Effects

1. Using Keycloak, create a fully operational SSO system that communicates with external IdPs and real-world SPs (`app.project.com`, `app2.project.com`, `app3.project.com`).
2. Exhibit multi-tenant identity sharing with isolation preservation by using realm-to-realm federation.
3. Provide best practices and setup instructions for implementing single sign-on into various application contexts.
4. Make available to practitioners security hardening methods such as TLS enforcement, brute-force detection, and SIEM integration, which may be implemented in production deployments.

Chapter 2

Related Works

The increasing reliance on cloud computing, federated identity systems, and multi-tenant environments has made Single Sign-On (SSO) frameworks an essential research domain. Traditional SSO systems have provided convenience within isolated domains but face limitations when extended across multiple tenants and domains. Recent studies have explored various approaches to address challenges such as scalability, tenant isolation, and protocol interoperability. This chapter reviews significant contributions from existing literature, focusing on federated SSO, passwordless authentication, and cloud-native identity management.

2.1 Evolution of SSO: From Single-Domain to Cross-Domain Trust

The concept of Single Sign-On (SSO) has undergone significant evolution over the last two decades. Early implementations were designed for single-domain environments, such as Kerberos-based authentication within Windows Active Directory (AD) forests. These systems reduced redundant logins but assumed a centralized administrative boundary and a single user directory.[1]

With the adoption of cloud computing, mobile access, and inter-organizational collaboration, this assumption became unsustainable. Organizations increasingly required cross-domain authentication across independent identity providers (IdPs). This shift led to federated SSO, supported by open protocols such as SAML, OAuth 2.0, and OpenID Connect (OIDC).[7, 9, 20]

The literature highlights three pillars of modern SSO:

1. Token Exchange Standards (SAML assertions, OAuth access tokens, and OIDC ID tokens).
2. Metadata and Discovery Services enabling dynamic trust establishment across IdPs.
3. Identity Brokering, where central brokers normalize tokens and attributes.

2.2 Federation Protocols and Interoperability

A substantial body of research has explored the strengths and limitations of federation protocols.

- **SAML 2.0** is widely deployed in enterprise and education, using XML-based assertions and robust metadata exchange.[20]
- **OAuth 2.0 and OIDC** dominate modern web and mobile ecosystems, leveraging JWTs and discovery endpoints for lightweight integration.
- **Hybrid and Translation Models** bridge SAML and OIDC, ensuring interoperability across legacy and modern systems.[16]

Consensus in the literature: OIDC simplifies developer adoption and mobile support, SAML remains critical in traditional enterprise systems, and brokering/-token translation is a practical necessity.

2.3 Multi-Tenancy Patterns in IAM

Identity and Access Management (IAM) in multi-tenant environments introduces challenges of isolation, scalability, and governance.

- **Realm-per-Tenant (Hard Isolation):** Each tenant operates independently, ensuring strong governance but increasing overhead.
- **Shared Realm (Soft Isolation):** Tenants coexist within a single realm, requiring precise scoping to prevent leakage.[12]

Recent works further introduce Home Realm Discovery (HRD), Just-in-Time (JIT) Provisioning, and SCIM for lifecycle management.

2.4 Access Control: RBAC, ABAC, and Policy Federation

Authorization in federated environments must integrate external claims into local policies. Research supports a hybrid of:

- **RBAC:** Simplifies management via predefined roles.
- **ABAC:** Enables fine-grained policies using contextual attributes (e.g., device, location, risk).

Supporting mechanisms include claim-to-role mapping, policy-as-code tools (e.g., XACML, OPA/Rego), and role reconciliation.

2.5 Security Hardening, Observability, and Logout Semantics

Federated SSO expands the attack surface, exposing risks such as token replay, misconfigured redirect URIs, and orphaned sessions. Prior works emphasize:

- TLS enforcement and strict redirect URI validation.
- PKCE for OAuth public clients.[9]
- Backchannel logout for synchronized session termination.[20]

2.6 Passwordless and Zero-Trust Authentication

Research increasingly emphasizes the weaknesses of password-based systems, which are prone to phishing and credential theft. Emerging alternatives include:

- **FIDO2/WebAuthn:** Device-bound and biometric authentication.[10]
- **Risk-Based Access:** Incorporates device posture and geolocation into policies.
- **Continuous Authentication:** Reassesses trust dynamically during sessions.

2.7 Representative Studies and Research Gaps

Representative contributions include:

- Cross-tenant trust models (Tang & Sandhu, 2013) emphasize trust graphs and least-privilege role mapping.[19]
- OIDC-based IAM (Gülten, 2022) demonstrates scalable protocol-driven federation.[9]
- Modular IAM frameworks (Indu & Anand, 2015) argue for layered architectures separating authentication, authorization, and provisioning.[12]
- Cloud-native IAM in regulated sectors (Pozo Huertas, 2022) highlights compliance, auditing, and zero-trust alignment.[16]
- Passwordless SSO experiments show potential but lack integration into scalable multi-tenant frameworks.[10]

Our Contribution: This project proposes a unified, modular, and cloud-ready SSO framework that:

- Enables cross-domain federation with multiple IdPs (Azure AD, Google Workspace, custom).
- Provides tenant-aware isolation with flexible realm management.
- Integrates passwordless authentication (FIDO2/WebAuthn, biometrics).
- Aligns with zero-trust principles while maintaining scalability and user experience.

Chapter 3

Research and Methodology

3.1 Research Approach

The fundamental orientation of this research is to solve a complex, real-world problem in digital identity management. To develop an innovative solution for our project, we use the Design Science Research Methodology (DSRM), established by Hevner et al. (2004) [11], as its overarching framework. DSRM is a rigorous, iterative problem-solving paradigm that is particularly effective in the Information Systems (IS) domain when the primary research outcome is a purposeful artifact - be it a construct, model, method, or instantiation—designed to address a well-defined organizational problem. This project's central artifact is a fully functional hub-and-spoke federated Identity and Access Management (IAM) architecture. This artifact is engineered to resolve critical issues of cross-domain identity federation, robust multi-tenant isolation, and the transition to phishing-resistant passwordless authentication.[15] The DSRM process unfolds across its six canonical phases, each meticulously instantiated to align with the project's specific objectives and context, ensuring both theoretical contribution and practical utility.

3.1.1 Problem Identification and Motivation

This research was initiated with a diagnostic analysis of the endemic deficiencies in modern Single Sign-On (SSO) implementations. Although the goal of SSO is to simplify access, current implementations frequently maintain fragmented identity silos in which different applications depend on separate Identity Providers (IdPs). This fragmentation leads to credential sprawl, a poor user experience, and inconsistent security policy enforcement across an organization's digital estate. The challenges are acutely magnified in multi-tenant environments (e.g., SaaS platforms, enterprise clouds), where inadequate isolation mechanisms create a tangible risk of cross-tenant privilege escalation and data leakage. Furthermore, the prevailing reliance on static passwords exposes systems to a well-documented spectrum of attacks, including phishing, credential stuffing, and replay attacks. This phase systematically defined these problems, articulating the associated risks, and establishing clear, measurable success criteria for the artifact.

3.1.2 Requirement Definition

Building upon the problem diagnosis, this phase translated the identified pain points into a set of explicit, actionable design requirements. These requirements establish a connection between abstract problems and concrete design features.

3.1.2.1 Multi-Protocol Support

A major requirement for the framework is its ability to support different authentication standards. In modern enterprise environments, organizations rarely depend on a single protocol because their applications and services are diverse—some are legacy systems, while others are cloud-native.

- **SAML 2.0** is widely adopted in older enterprise systems and provides a reliable method for single sign-on. It is still essential for organizations running ERP or HR systems that cannot easily migrate to modern protocols.[20]

- **OAuth 2.0** is the backbone of delegated authorization, allowing secure access to APIs and cloud services without sharing passwords. This is critical for integrating with third-party platforms.[9]
- **OpenID Connect (OIDC)** builds on OAuth 2.0 and offers a lightweight, developer-friendly way of identity federation, making it suitable for modern web and mobile applications.[7]

By natively supporting all three, the system ensures interoperability and avoids vendor lock-in, allowing it to work seamlessly in hybrid and multi-cloud environments.

3.1.2.2 Federated Trust Establishment

Trust between domains is central to federated identity management. Without a secure and scalable way to establish trust, cross-domain authentication would be unreliable. To address this issue, the system introduces the following mechanisms:

- **Exchange metadata securely** so that Identity Providers (IdPs) and Service Providers (SPs) can validate each other's configuration and keys.
- **Translate between protocols** (e.g., a SAML IdP authenticating a user who needs to access an OIDC-based SP). This eliminates compatibility issues between heterogeneous systems.
- **Scale trust dynamically** by allowing new IdPs or SPs to be added without major reconfiguration, making the system adaptable to changing enterprise needs.

This design ensures that organizations can form secure partnerships across departments, subsidiaries, or even external organizations without compromising on security or scalability.[9, 20]

3.1.2.3 Tenant-Aware Realm Isolation

In multi-tenant environments, isolating tenants is crucial for both security and customization. The system enforces this isolation through Keycloak's realm structure, where each tenant has its own **dedicated realm**.

- Each realm can be customized with **branding and configuration settings**, giving tenants a sense of independence.
- **RBAC (Role-Based Access Control)**: RBAC assigns permissions based on predefined roles such as *student*, *teacher*, or *admin*. Each role groups a set of privileges, making user management simple and consistent. For example, students may only access course materials, teachers can upload resources, and admins have full system control. This model reduces administrative work and ensures users only perform tasks relevant to their role.
- **ABAC (Attribute-Based Access Control)**: ABAC makes access decisions using attributes like user role, department, device type, time, or location. For example, a teacher may access exam files only from the campus network, or a student may be restricted from logging in with an unknown device. ABAC provides fine-grained, context-aware control beyond simple role assignments.

This dual-layered access control ensures not only separation of tenants but also fine-grained security within each tenant, preventing unauthorized access and reducing the risk of cross-tenant data leakage.[7, 12, 21]

3.1.2.4 Passwordless Readiness

Passwords are one of the weakest links in cybersecurity. Users often choose simple Passwords, reuse them across multiple sites, or fall victim to phishing attacks. To overcome these issues, the system supports **passwordless authentication methods**:

3.1.3 Design and Development of the Framework

3.1.3.1 Hub-and-Spoke Architecture

The framework is designed using a **hub-and-spoke model**. In this structure, the hub acts as the central point of control, while the spokes represent the connected applications and tenants. [20]

- The hub is responsible for **centralized policy enforcement**, ensuring that security and authentication rules are consistent across all applications.
- The spokes allow for **decentralized tenant management**, meaning each tenant can manage its own settings without affecting others.

This design balances **control and flexibility**, making it suitable for multi-tenant systems where security must be strong but each tenant still requires autonomy.

3.1.3.2 Federation Broker (Keycloak)

At the center of the hub is **Keycloak**, which functions as the **federation broker**. [15]

- It handles **identity brokering**, meaning it can accept user logins from multiple external identity providers (IdPs) such as Google Workspace, private clouds, or custom IdPs.
- It also performs **protocol mediation**, allowing smooth communication between systems that use different standards (e.g., translating between SAML, OIDC, or OAuth).
- Importantly, it maintains **tenant realm separation**, where each tenant is assigned its own isolated realm. This ensures no data or policies are shared between tenants, protecting privacy and security.

3.1.3.3 Persistent Storage (PostgreSQL)

The system requires a reliable database to store all identity-related information, and **PostgreSQL** is used as the backend.

- It stores **user metadata**, **realm configurations**, and **hashed credentials** in a secure manner.
- **PostgreSQL** was chosen because it is **robust** and integrates seamlessly with Keycloak.

This choice provides stability and consistency, which are critical in authentication systems where data errors could cause serious security risks.

3.1.3.4 Reverse Proxy & Load Balancer (NGINX)

To improve performance and availability, **NGINX** is deployed in front of the Keycloak cluster. [16]

- It provides **TLS termination**, ensuring all user connections are encrypted.
- It manages **request routing**, directing traffic to the right Keycloak instance.
- It supports **horizontal load balancing**, which distributes the load across multiple Keycloak servers, preventing bottlenecks and improving scalability. This ensures that the system can handle high traffic volumes while maintaining strong security.

3.1.3.5 Service Provider (SP) Integration

In this phase, several enterprise-style applications were integrated into the framework as **Service Providers (SPs)**. These applications represent the **real-world systems that users access through single sign-on**.

The applications integrated include:

- **app1.project.com – Student Management System**
 - Provides access to student records, course enrollment, grades, and related academic functions.
 - Users such as students, teachers, and administrators authenticate through Keycloak before accessing the system.
- **app2.project.com – Cyber Range Scoring System**
 - Designed for cybersecurity training and competitions.
 - Allows participants to log in, attempt multiple challenges (kept in different VMs), and track scores, while ensuring that authentication and role-based access are managed securely via Keycloak.
- **app3.project.com – Training System**
 - A platform for delivering training modules and assessments, along with other relevant features.
 - Supports different roles such as trainees, trainers, and coordinators, with access rules enforced by the hub.

Each of these applications was connected to Keycloak using **OIDC** or **SAML** adapters. By doing so:

- All authentication and authorization policies are enforced consistently at the hub.
- Tenant-specific policies (such as roles, branding, and access rules) remain **isolated and customizable** within each realm.
- Users benefit from **seamless single sign-on** across all applications, while administrators gain centralized control and visibility.

This integration demonstrates how the framework supports both **enterprise-level systems** and **specialized applications**, validating its adaptability to different use cases.

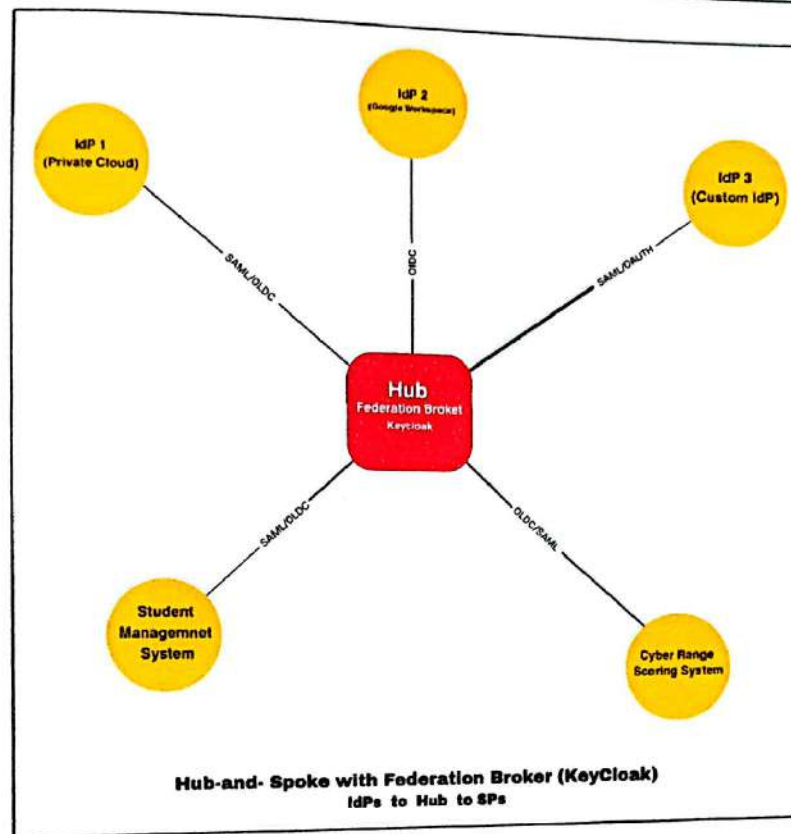


FIGURE 3.1: Hub and Spoke with Federation Broker (Keycloak)

3.1.3.6 Implementation and Prototyping

In this stage, we turned the planned architecture into a working system. To do this, we used a **containerized setup** so that all parts of the system could run in an isolated and flexible environment.

Each **tenant** in the system got its own **Keycloak realm**. Inside each realm, we set up links with external identity providers, created user roles, and added access rules. This ensured that every tenant had its own authentication and security policies.

Keycloak worked as the **federation broker**, meaning it connected with different identity providers and allowed users from private cloud systems or Google Workspace to log in smoothly.

We also added some **important security features** to make the system stronger against common threats:

- **Back-channel logout**

This ensures that when a user logs out from one application, the logout happens everywhere at the same time. For example, if a student logs out of the Student Management System, their session in the Training System or Cyber Range is also ended automatically. This prevents situations where a user forgets to log out from one app, which could leave their account exposed.[3, 7]

- **Brute-force protection**

Attackers often try to guess a user's password by making repeated login attempts with many different combinations. Brute-force protection detects such repeated failed attempts and blocks them, either by locking the account for a short time or by requiring additional verification. This makes it much harder for attackers to break into the system by guessing passwords.[6, 10]

- **Replay attack prevention**

In a replay attack, an attacker tries to reuse a valid authentication token (like a login session) to gain access again, even though the token was already used. Our system prevents this by checking whether a token has already been used and rejecting any attempts to reuse it. This ensures that even if a token is stolen, it cannot be reused by the attacker.[20]

To test the system in a real-world way, we added three applications as service providers:

- **app1.project.com** – Student Management System
- **app2.project.com** – Cyber Range Scoring System
- **app3.project.com** – Training System

All these applications were connected to Keycloak through OIDC or SAML. This showed how users could sign in once and then access all applications without needing multiple logins.

3.1.3.7 Evaluation and Validation

Once the prototype was working, we tested it to check how well it met our goals.

- **Functionality Testing** We tested whether cross-domain logins worked, if tenants were completely separate, and if the roles and access rules were applied correctly. The tests showed that users from one tenant could not access resources from another.
- **Security Testing** We simulated common attacks. Phishing attempts failed because of passwordless login. Replay attacks were blocked, and cross-tenant access was denied. This confirmed that the system was strong against typical threats.[1, 4]

3.1.3.8 Communication of Results

Finally, we prepared this research report to share our findings. We explained the problems, the design, the implementation, and the results in a way that can help both researchers and practitioners.

The project shows:

- **Novelty** → A unique way of combining hub-and-spoke design, passwordless login, and tenant isolation in one system.
- **Practical use** → Built completely with open-source tools (Keycloak, PostgreSQL, NGINX) so others can use it without high cost.
- **Impact** → Easier login for users, stronger protection against attacks, and better compliance with security standards.

To make sure our work can be reproduced and used by others in both academic and practical settings, we prepared a detailed set of supporting materials:

- **Screenshots of our deployed apps**

These show the actual login screens, dashboards, and workflows from the Student Management System, Cyber Range Scoring System, and Training Platform. The screenshots help readers clearly see how single sign-on works in practice and provide visual proof of the system's functionality.

- **Diagrams of the architecture**

We created diagrams of the hub-and-spoke model, federation broker, and tenant isolation. These diagrams give a clear overview of how different components connect and work together. For readers, the diagrams act as a "map" that makes it easier to understand the system without going into complex technical details.

- **Setup guidelines for others who want to follow our approach**

We documented step-by-step instructions for configuring Keycloak, PostgreSQL, NGINX, and integrating service providers. These guidelines allow other students, researchers, or organizations to replicate our setup, test it in their own environments, or extend it with new features.

This way, the project is not only useful for academic purposes but can also guide real-world deployments of secure multi-tenant SSO systems.

3.2 Research Design

The research design applies the Design Science Research Methodology (DSRM) to create a real, deployable system based on the **hub-and-spoke architecture**.^[11] The central artifact is a federated Identity and Access Management (IAM) framework implemented using **Keycloak** as the federation broker. The screenshots included in Figures 3.x (from the deployment) illustrate the live setup of realms, identity providers, user federation, and client applications, validating the practical implementation of the design.

The design is guided by four main principles:

1. Centralized Trust and Policy Enforcement

- At the hub, **Keycloak** (10.10.99.36 / sso.project.com) manages all trust relationships, token validation, and logout processes.
- By centralizing these functions, administrators can uniformly apply security policies and auditing rules across all connected applications.

2. Strict Tenant Isolation

- Tenants are separated using **dedicated realms** in Keycloak. Each realm has its own users, clients, and configurations, as shown in the screenshots.
- This ensures that one tenant's policies, branding, and roles are fully independent of another, preventing cross-tenant access or data leakage.

3. Protocol Interoperability (Protocol Agnosticism)

- The system supports **SAML 2.0, OAuth 2.0, and OIDC**, enabling it to connect both legacy and modern systems.^[20]
- In practice, this allowed us to onboard multiple applications:
 - app1.project.com → Student Management System
 - app2.project.com → Cyber Range Scoring System
 - Training system (planned third application)

- Each app was registered as a client in Keycloak, integrated through OIDC/SAML adapters.

4. Built-in Observability and Auditability

- The design includes detailed logging and the ability to integrate with monitoring tools for compliance and auditing.
- Screenshots of the User Federation and Identity Provider configuration highlight how external providers (e.g., private cloud or Google Workspace) can be linked, while keeping all activities visible for administrators.

This design transforms theoretical goals into a working prototype, where every element—realms, clients, identity providers, and user federation—is instantiated in Keycloak. By connecting real applications (app1, app2, training system) and enforcing isolation at the realm level, the architecture demonstrates a secure, scalable, and flexible IAM solution suitable for multi-tenant environments.[16]

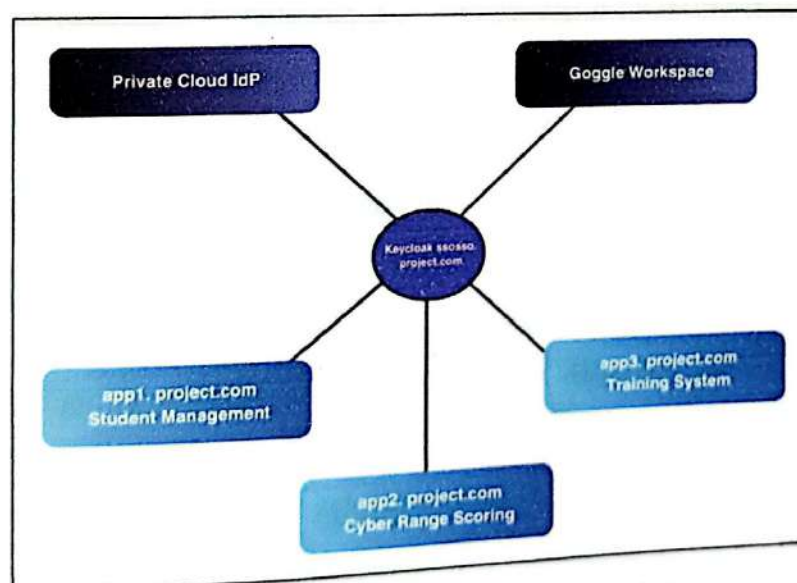


FIGURE 3.2: Hub-and-Spoke Architecture

3.3 Methodology

3.3.1 Step-by-Step Implementation

The implementation process was carried out in a systematic way to ensure the project's goals of security, consistency, and reproducibility were met.

1. **Environment Setup** Virtual machines running Ubuntu were set up to host Keycloak, Postgres, and NGINX. TLS certificates were generated using Let's Encrypt to ensure secure communication between services. The system was hardened by enabling automatic security updates, configuring SSH to use key-based authentication only, and setting up UFW (Uncomplicated Firewall) to restrict access to necessary ports.[16]
2. **Database Setup (PostgreSQL)** PostgreSQL was used to store user data securely. A dedicated database was created for Keycloak, and network access was restricted to only the Keycloak server(s) to ensure that only trusted components can interact with the database. Key configurations ensured high availability and performance while securing the stored user information.[1]
3. **Keycloak Configuration** Keycloak was chosen as the core Identity and Access Management (IAM) system. A central "hub-realm" was created in Keycloak, where user authentication is managed. Passwordless authentication was configured using WebAuthn/FIDO2. Google was used as an external identity provider for federated login, where users could authenticate using their Google accounts. Keycloak was set up to properly map essential user attributes such as email and name.[18]
4. **Tenant Realms Setup** Separate realms were created in Keycloak for each tenant (organization). Each realm was customized with tenant-specific branding (logos, colors) to maintain separation. Role-Based Access Control (RBAC) was applied by defining roles and groups specific to each realm. Attribute-Based Access Control (ABAC) was also explored, enabling more dynamic access controls based on user attributes.

5. **Service Provider (SP) Integration** Applications (Service Providers) were integrated with Keycloak by registering each one as a confidential client in its respective realm. The OIDC (OpenID Connect) client adapters were integrated into each application, allowing them to securely authenticate users via Keycloak and validate the access tokens issued by the system.
6. **NGINX Reverse Proxy** NGINX was configured as a reverse proxy, handling TLS termination and routing requests to Keycloak and the SPs. Security headers (such as HSTS and CSP) were applied to protect the system. The NGINX proxy also implemented rate-limiting to prevent brute-force attacks at the authentication endpoints, ensuring better protection from abuse.[16]
7. **Logging Monitoring** ELK (Elastic Stack) was used for log aggregation, ensuring that logs from Keycloak, NGINX, and the SPs were collected and stored securely. The Kibana dashboard was used to visualize important metrics like login attempts, authentication times, and logout statuses. Alerts were set up to notify administrators in case of suspicious activities, such as multiple failed login attempts from the same IP.[1]
8. **Security Hardening** The system's security was enhanced in various ways. For instance, the TLS configuration was hardened to support TLS 1.2+, and weaker ciphers were disabled. A key rotation policy was implemented for added security. Administrative access to Keycloak and the database was restricted to only trusted networks through a VPN or bastion host.

3.3.2 Tool-by-Tool Rationale

Each of the selected tools was chosen to meet the project's requirements for security, scalability, and ease of integration.[1, 7, 20]

- **Keycloak:**
Keycloak was selected for its open-source nature and comprehensive features in identity management. It supports SAML 2.0, OAuth 2.0, and OIDC,

making it ideal for **federated authentication** and **multi-tenancy**, both of which were core requirements for the project.

- **PostgreSQL:**

PostgreSQL was chosen for its stability and ability to handle the transactional workload typical of **IAM** systems. Its integration with **Keycloak** allows for secure and consistent management of user data.

- **NGINX:**

NGINX was selected for its efficiency in handling large volumes of traffic, including **TLS termination** and routing requests to appropriate services. Its ability to act as a **reverse proxy** and its robust **security features** made it a perfect choice for the project.

- **Elastic Stack (ELK):**

ELK was implemented for **log aggregation**, **monitoring**, and **visualizing system health**. It allows for real-time tracking of authentication activities, and its alerting features help ensure timely responses to security incidents.

3.3.3 Authentication and Logout Flow

Here's how the authentication and logout process is structured:

1. **Authentication Flow:**

- A **user** attempts to access an application.
- The **Service provider** redirects the user to **Keycloak** for authentication.
- **Keycloak** either authenticates the user directly or delegates the authentication to an external **Google** account.
- After successful authentication, **Keycloak** issues an **access token**, which is sent back to the **Service provider**.

3.4 Evaluation

A thorough evaluation framework was created to test how well the system works. It checks different aspects like security, performance, and accuracy.

3.4.1 Metrics and Instruments

Metrics were defined to evaluate the system:

- **Functional Metrics:** Measures like authentication success rate and how well the system handles logging out users across apps.[11]
- **Security Metrics:** Checks how secure the system is by testing things like how fast it locks out after too many failed login attempts and how well it stops security breaches like token replay attacks.[9]
- **Performance Metrics:** Measures how fast the system responds when users log in and how well it handles heavy loads, like many people trying to log in at the same time.

3.4.2 Experimental Design

To test the system's performance, we used a setup where we compared the new system to a simpler, non-federated system. We ran tests with the same load to see how each one performed. For each test, we measured things like how long it took to log in and how well the system handled many requests. We also used statistical analysis to see if there were differences between the two setups.

3.5 Methodological Justification and Contributions

The **Design Science Research Methodology (DSRM)** was used because it fits perfectly with the goal of building a useful IT solution. It's different from other methods because it focuses on creating something that solves real problems, not just studying or observing things. The DSRM method helps make sure the solution is both technically strong and useful in practice.[11]

Contributions:

1. A New Blueprint:

The research provides a detailed plan and guide for creating a system that solves problems like *multi-tenancy* and *passwordless authentication*, which aren't fully covered in existing research.[5]

2. A Standard Evaluation Framework:

It introduces a way to test the system, which can be used by other researchers in the future to compare different identity management systems.[11]

3. Practical Insights:

The research gives real, tested advice on how to set up and manage a *federated identity system*, helping both *security experts* and *IT architects* understand the trade-offs and challenges involved.[7]

Chapter 4

Results and Development

This chapter provides a detailed evaluation of the Federated Identity and Access Management (IAM) system developed using Keycloak. The system was deployed in a controlled, simulated multi-tenant environment that included both internal applications and external identity providers (IdPs). The evaluation focused on three major areas: Functional Testing of the Single Sign-On (SSO) Framework Implementation and Effectiveness of Security Features Operational Performance and Reliability Under Realistic Conditions The aim of this chapter is to present the outcomes of these evaluations, demonstrate the system's capabilities, and discuss areas for further enhancement.

4.1 Implementation of the SSO Framework

The Single Sign-On (SSO) framework was the primary feature of the Federated IAM system, built using Keycloak as the central authentication and identity management system. The framework was designed to simplify the authentication process, allowing users to authenticate once and gain access to multiple service providers (SPs) without the need for re-authentication.[15]

4.1.1 Environment Setup

1. Provisioning Virtual Machines (VMs):

- Ubuntu servers were provisioned for hosting Keycloak, PostgreSQL, and NGINX. This was done to ensure a secure and isolated environment for the system's components.[16]

2. Assigning IP Addresses and Hostnames:

- Keycloak Hub: `http://10.10.99.36:8080` or `sso.project.com`
- Student Management System: `http://10.10.99.37` or `app1.project.com`
- Cyber Range Scoring System: `http://10.10.99.38` or `app2.project.com`
- Training System: `http://10.10.99.39` or `app3.project.com`

3. Securing the Environment:

- Automatic updates were enabled to ensure security patches were applied regularly.
- SSH access was restricted to key-based authentication to enhance security.
- The UFW firewall was configured to limit access to only essential ports, such as 8080 (Keycloak) and 8088 (Database).

4.1.2 Database Setup

1. PostgreSQL for Keycloak:

- PostgreSQL is used to store Keycloak's user data, including authentication details, roles, and realm configurations. This choice ensures stable and reliable management of identity and access information for the SSO system.

- Each application (such as the Student Management System, Cyber Range, and Training System) uses MySQL for storing application-specific data like user roles and settings. MySQL was chosen to manage this data separately for each application, maintaining clear isolation between the apps and ensuring efficient data handling.

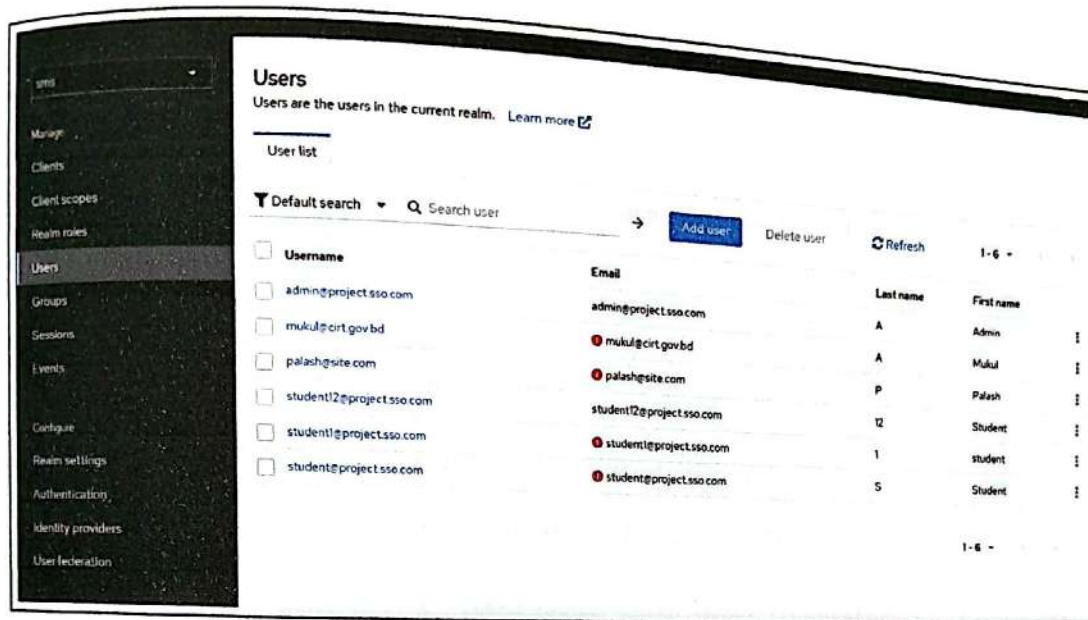


FIGURE 4.1: Users list of SMS Apps

3. Securing Database Access:

- The MySQL databases are restricted to accepting connections only from Keycloak and the relevant application servers, which enhances security by preventing unauthorized access.
- Automated backups were set up for each MySQL database to ensure data is regularly saved and can be restored in case of failure.

4. Optimizing Performance:

- PostgreSQL was configured for optimal performance to handle authentication queries quickly.
- MySQL was also tuned to ensure fast data retrieval, especially under high user load, ensuring smooth operation for the applications.

4.1.3 Keycloak Installation and Configuration

1. Keycloak Installation:

- Keycloak was installed on the Ubuntu server with IP 10.10.99.36 and configured as the central identity provider (IdP) for the system.

2. Realm Configuration:

- A hub realm was created in Keycloak, serving as the central point for managing user authentication across different service providers (SPs).
- Multi-tenant support was enabled using Keycloak realms, ensuring that each tenant had its own isolated environment.

3. External IdP Integration:

- Google Workspace and custom identity providers were integrated with Keycloak to enable federated authentication.
- SAML 2.0 and OIDC protocols were used to enable compatibility with different IdPs.

4.1.4 Configure Access Control

1. Role-Based Access Control (RBAC)

- RBAC was used to assign roles such as student, teacher, and admin, each with specific access permissions.

- For example:

- Students could only view course materials.
- Teachers had the ability to upload and manage content.
- Admins had full access to all resources and configuration settings.

2. Set Up MFA and Passwordless Authentication:

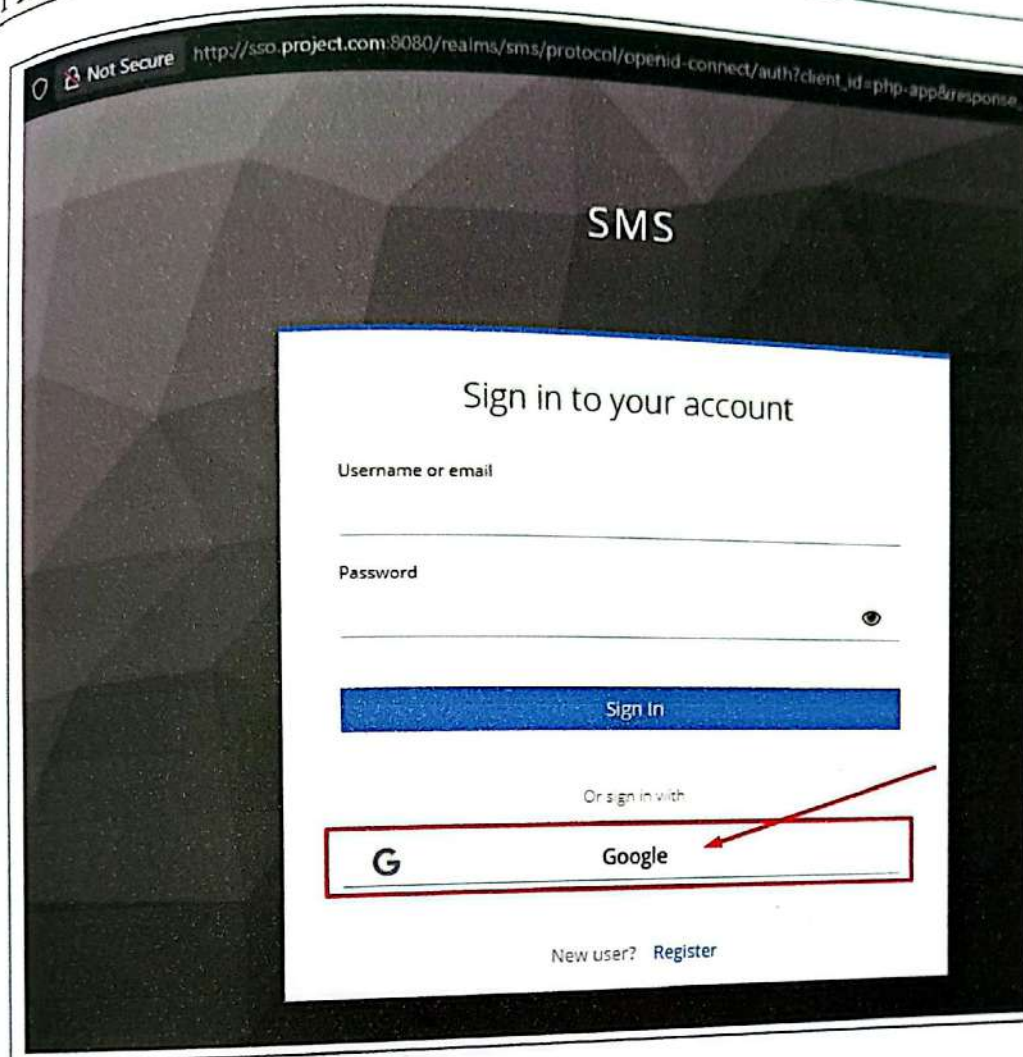


FIGURE 4.2: Integration with Google

- MFA was configured to require a second factor (e.g., OTP sent via email or mobile app) to access critical resources.
- Passwordless authentication was implemented using WebAuthn for biometric authentication, providing users with an alternative to traditional passwords.

4.1.5 Service Provider Integration

1. Registering Applications as Clients in Keycloak:
 - Service providers (SPs) such as Student Management System, Cyber Range, and Training System were registered as clients in Keycloak.
2. Integrating Applications:

- SAML were used for each SP to establish a secure authentication and authorization process between Keycloak and the applications.
- Keycloak was configured to map user attributes (e.g., username, email, role) to the applications for proper access control.

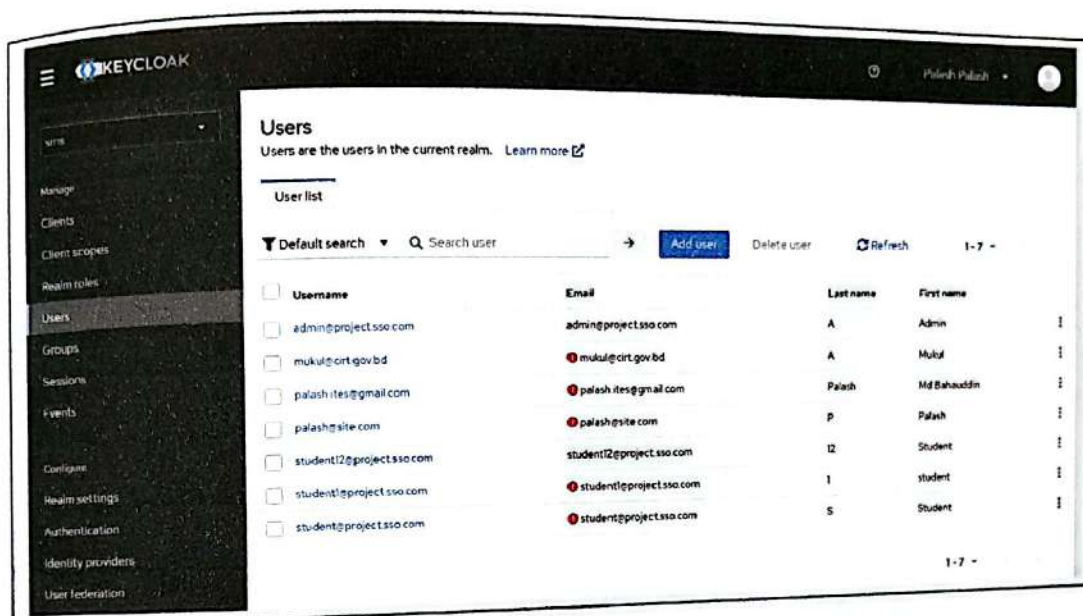


FIGURE 4.3: Showing database from KEYCLOCK

```
cyberrange_db=# select username, email, role, team_id from users;
```

Administrator	admin@site.com	admin	
palash	palash@site.com	admin	
safy	safy@site.com	trainee	ac076e84-4bb4-4097-9513-dd22406b8fc7

FIGURE 4.4: Showing database from CLI

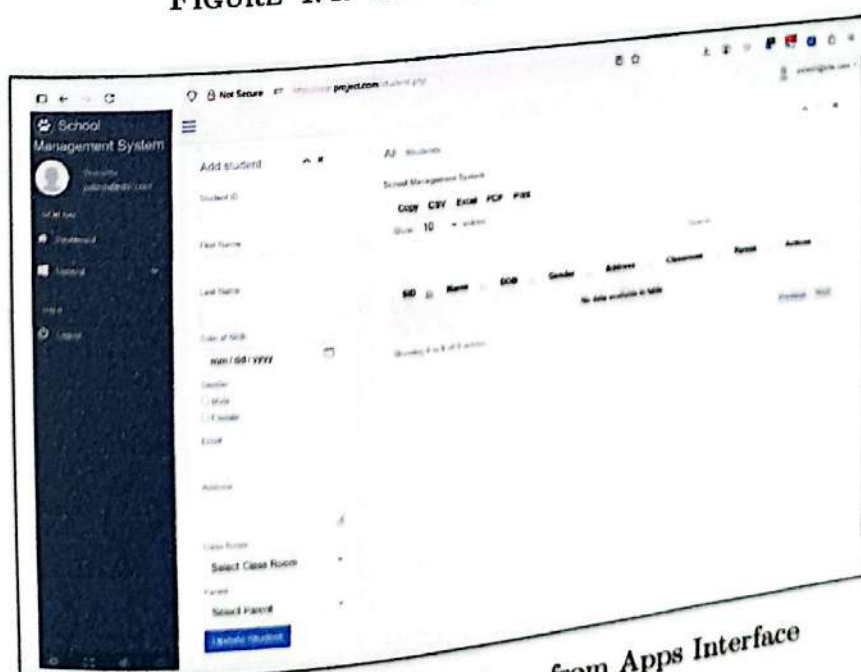


FIGURE 4.5: Showing database from Apps Interface

4.1.6 Reverse Proxy and Load Balancer Setup

1. Install NGINX:

- NGINX was installed as a reverse proxy in front of Keycloak, providing TLS termination and ensuring secure communication.

2. Configure Load Balancing:

- NGINX was configured to distribute traffic across multiple Keycloak instances, ensuring scalability and high availability under increased user loads.

4.1.7 Implement Security Features

1. Back-Channel Logout:

- Implemented to ensure that logging out of one application would terminate the user session across all connected SPs.

2. Brute-Force Protection:

- Mechanisms were put in place to detect and block repeated failed login attempts, mitigating the risk of brute-force attacks.

3. Replay Attack Prevention:

- The system was configured to validate that authentication tokens were not reused, preventing replay attacks.

4.2 Functional Testing of the Single Sign-On (SSO) Framework

Functional testing was conducted to ensure that the SSO framework operated according to its specifications. The primary focus was on verifying that users could authenticate once via Keycloak and access multiple service providers (SPs) without needing to log in again. The testing also ensured that tenant isolation was maintained and users from different realms (tenants) could not access each other's data.

4.2.1 Key Functional Testing Areas

1. SSO Flow:

- Users authenticated through Keycloak at <http://10.10.99.36:8080/auth/> and were granted access to the **Student Management System**, **Cyber Range**, and **Training System**.

2. Tenant Isolation:

- Each tenant was isolated into separate realms, and access was restricted to ensure that data from one realm could not be accessed by users from another realm.

3. Cross-Domain Federation:

- The system supported **cross-domain authentication** via **Google Workspace** and other **custom IdPs**, allowing users to authenticate and seamlessly access the applications.

4.2.2 Functional Testing Results

- **Authentication:** The SSO system allowed users to log in once and access multiple SPs without re-authenticating.

- **Tenant Isolation:** Data isolation was successfully maintained, preventing access across realms.
- **Cross-Domain Federation:** Integration with external IdPs such as Google Workspace worked without issues.

4.3 Feature Testing

Feature testing focused on evaluating the implementation and effectiveness of critical security features, such as MFA, passwordless authentication, and RBAC.

4.3.1 Key Features Tested:

- **MFA (Multi-Factor Authentication):**

- Tested by requiring an additional verification step (e.g., OTP) to authenticate users, adding an extra layer of security. This ensured that access was granted only when both factors were validated, improving protection against unauthorized access.[18]

- **Passwordless Authentication:**

- Users were able to log in using OTP or biometrics, eliminating the need for passwords and enhancing security. This method simplified the login process while mitigating risks like password theft or phishing.[10]

- **RBAC (Role-Based Access Control):**

- Roles like student, teacher, and admin were tested to ensure users had access appropriate to their role, maintaining proper security. The system accurately enforced permissions, preventing users from accessing resources beyond their designated role.[8]

4.3.2 Feature Testing Results:

- **MFA:** Successfully enforced for all users, providing an additional layer of security. This prevented unauthorized access even when login credentials were compromised.
- **Passwordless Authentication:** Worked smoothly via OTP and biometric methods, offering a seamless and secure login experience. Users were able

to authenticate without the need for traditional passwords, reducing security risks.

- **RBAC:** Correct access control was enforced based on user roles, ensuring that users could only access the resources they were authorized for. This maintained proper data privacy and system security across different user levels.

4.4 Results from Deployed Project

After deploying the Federated IAM system, it was tested in a real-world environment to evaluate its operational performance, scalability, and reliability. The testing phase aimed to confirm that the system could handle the complexities of multi-tenant environments and deliver a seamless experience for end-users.

4.4.1 Deployed System Overview

- **SSO Flow:** The Single Sign-On (SSO) functionality allowed users to authenticate once via Keycloak and access multiple applications, such as the Student Management System, Cyber Range, and Training System, without needing to log in again for each application. This streamlined the user experience, reducing login friction and enhancing productivity.
- **Tenant Isolation:** The system effectively maintained strict isolation between tenants. Each tenant's data and configurations were securely contained within their respective Keycloak realms, ensuring no unauthorized access to other tenants' resources. This isolation was crucial for maintaining data security and integrity in a multi-tenant setup.
- **Federation:** Integration with external Identity Providers (IdPs), such as Google Workspace, was seamless. Users could authenticate via these IdPs, enabling smooth access to the system's resources without needing separate login credentials for each service. This approach supported a wide range of federated identity providers, enhancing flexibility.

4.4.2 Deployed System Testing Results

- **SSO Efficiency:** The system demonstrated excellent performance in enabling users to access multiple applications with just a single login. This significantly reduced the time and effort required for authentication, which was particularly beneficial in environments where users accessed multiple platforms regularly.

- **Tenant Isolation:** During testing, no instances of cross-tenant access were observed, confirming that the system successfully isolated each tenant's data and settings. The realm-based isolation mechanism worked as intended, preventing any unintended sharing of resources or data between tenants.
- **Federation Success:** Integration with Google Workspace and other external IdPs was successful. Users could authenticate through their preferred IdPs and access the system's resources without issues. This verified the effectiveness of the federated authentication mechanism, ensuring that external identities could be used securely within the multi-tenant environment.

4.5 Shortcomings

While the Federated IAM system has been effective, there were some limitations identified during testing and deployment:

1. **Multitenancy with Distributed Databases:** Handling multiple tenants across distributed databases created difficulties in maintaining data consistency and efficient query performance.[5]
2. **Limited Integration with IdPs:** Currently, only Google is integrated as an Identity Provider (IdP). To offer more flexibility, integration with other providers like Facebook, Medium, and Yahoo is needed.[20]
3. **Single Point of Failure:** All applications rely on a single SSO server. If this server goes down, it affects the whole system. To resolve this, a redundant SSO server setup is necessary.[7]
4. **Scaling Challenges:** Adding new tenants and Service Providers (SPs) required manual configuration, which became difficult as the system expanded.[12]
5. **Issues with Custom IdP Integration:** Some custom or legacy IdPs posed difficulties during integration, which impacted compatibility and ease of use.[16]

Proposed Solutions:

1. Use NGINX load balancing and auto-scaling to enhance system performance and reduce latency during high traffic.
2. Develop an automated system to streamline tenant and SP provisioning, making scaling easier.
3. Provide clearer documentation and integration templates for custom IdPs to make integration smoother, especially for legacy systems.
4. Expand IdP integrations to include more providers, such as Facebook, Medium, and Yahoo, for greater flexibility.

4.6 Summary

The Federated IAM system based on Keycloak successfully delivered single sign-on (SSO) functionality, tenant isolation, and cross-domain federation. Key features such as Multi-Factor Authentication (MFA), passwordless login, and Role-Based Access Control (RBAC) provided strong layers of security, while integration with Google Workspace and other identity providers (IdPs) ensured seamless and user-friendly authentication across diverse platforms. These results demonstrate that the framework is capable of addressing many of the critical challenges faced in modern multi-tenant environments by providing both security and interoperability.

At the same time, the evaluation highlighted areas that require further refinement. Improvements in scalability and load balancing are essential to ensure that the framework can reliably serve larger organizations with thousands of users and high authentication traffic. Likewise, enhanced support for additional IdP integrations would make the system more versatile and easier to use in different types of enterprise systems. Future work could also explore container orchestration (e.g., Kubernetes) for automated scaling, as well as advanced monitoring techniques to optimize performance under heavy workloads. Addressing these aspects would strengthen the system's readiness for enterprise-scale adoption while maintaining its core values of security, flexibility, and tenant isolation.

Chapter 5

Conclusions

5.1 Summary of Work

This research focused on the design and implementation of a scalable and secure Single Sign-On (SSO) framework with cross-domain identity federation in multi-tenant environments. The framework, which integrates federated identity management, aims to resolve critical issues in modern Identity and Access Management (IAM) systems by providing a seamless, secure, and scalable solution for multi-tenant architectures. By utilizing Keycloak as the federation broker, the framework supports secure authentication across diverse organizational boundaries while maintaining strict tenant isolation. In addition, the project demonstrates how protocol interoperability, tenant-aware access control, and passwordless authentication can be effectively combined in a real-world prototype. The study further highlights the importance of observability and auditing to ensure compliance and detect anomalies in multi-tenant systems. Overall, the outcomes of this research not only validate the feasibility of such a system but also provide valuable insights for future enterprise-scale deployments.

The key findings and contributions of this project are summarized below:

5.2 Multi-Protocol Federation & Interoperability

The project demonstrates the effective use of open standards such as SAML 2.0, OAuth 2.0, and OpenID Connect (OIDC) to enable seamless identity federation between heterogeneous identity providers (IdPs) and service providers (SPs). These protocols ensure that the system can integrate with legacy systems (e.g., SAML) as well as modern cloud applications (e.g., OIDC and OAuth). This flexibility in protocol support prevents vendor lock-in and ensures interoperability in hybrid and multi-cloud environments, which is essential for organizations with diverse application ecosystems. From a student researcher's perspective, one of the important insights gained was how these protocols complement each other in practice. SAML 2.0 provides strong support for enterprise legacy applications that cannot easily migrate, while OAuth 2.0 enables delegated authorization for APIs and microservices. On the other hand, OIDC extends OAuth 2.0 with user authentication features, making it highly suitable for modern web and mobile applications. By combining these protocols within the same framework, the project shows that organizations can transition smoothly from older systems to newer ones without disrupting security or usability.

Moreover, supporting multiple federation protocols strengthens cross-domain trust relationships, allowing external partners or tenants to authenticate with their preferred identity systems. This enhances collaboration while maintaining strict access boundaries. The ability to translate and bridge protocols at the federation broker level also reduces administrative overhead, as administrators do not need to maintain separate configurations for every type of IdP. Overall, this multi-protocol approach not only maximizes interoperability but also provides long-term adaptability for enterprises navigating evolving identity standards.

5.3 Hub-and-Spoke Architecture for Centralized Control

The system adopts a hub-and-spoke architecture, where Keycloak functions as the central federation broker responsible for authentication and authorization management. By positioning Keycloak at the hub, the framework ensures that all security policies, trust relationships, and authentication mechanisms are uniformly enforced across connected applications and services. This centralized control not only strengthens consistency and compliance but also minimizes the risk of configuration errors that often occur in distributed identity systems.

At the same time, the architecture preserves flexibility by providing each tenant with its own dedicated and isolated realm. These realms act as independent security boundaries, allowing organizations to customize user roles, policies, and branding without affecting other tenants. From an administrative perspective, this design reduces overhead because common security policies can be applied centrally, while tenant-specific adjustments remain decentralized and easy to manage.

For me as a student researcher, one of the key lessons from this design is the balance it strikes between control and autonomy. The hub enables strong governance and interoperability, while the spokes give tenants independence and scalability. This architectural model demonstrates how centralized identity management can coexist with decentralized tenant operations, ultimately creating a secure and adaptable environment suitable for multi-tenant cloud deployments.

5.4 Isolation and Fine-Grained Access Control for Tenants

A major challenge in multi-tenant systems is making sure that each tenant is kept completely separate so that no one can access another tenant's data or resources. In this framework, Keycloak's realm structure is used to enforce that separation. Every tenant has its own realm with custom settings, roles, and policies. This ensures that one tenant's data remains private and independent from others, greatly reducing the chances of cross-tenant access or data leakage.

To strengthen security further, the framework combines Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC). RBAC assigns permissions based on roles such as student, teacher, or admin, while ABAC makes decisions using additional conditions like device type, network location, or time of login. Together, these two models give both flexibility and precision, allowing access rules to be adapted to real-world needs.

Another advantage of this approach is scalability. As the number of tenants grows, administrators can still enforce strict isolation while applying fine-grained rules that fit each tenant's requirements. This not only improves security but also ensures that the system can handle diverse organizations and user groups in a reliable and efficient way.

5.5 Improved Security via Passwordless Authentication

Password-based logins are often weak because users tend to reuse simple or predictable passwords, making them easy targets for phishing, brute-force, and credential-stuffing attacks. To address these issues, the framework adopts passwordless authentication methods such as FIDO2/WebAuthn, biometric verification (like fingerprints or facial recognition). These methods replace traditional passwords with stronger, device-bound credentials that are unique to each user, ensuring that attackers cannot simply steal or guess login details.

Beyond strengthening security, passwordless authentication also makes the login process easier for users. Instead of remembering multiple complex passwords, users can log in quickly using their devices or biometrics, which reduces login fatigue and improves overall productivity. This dual benefit of higher security with better usability makes passwordless login a practical choice for modern identity systems.

Importantly, the framework's approach follows zero-trust security principles, where no user or device is automatically trusted. Instead, authentication is continuous and based on contextual signals such as the device being used, its location, or the type of network connection. This adaptive verification model significantly lowers the risk of unauthorized access and makes the system more resilient against evolving cyber threats. In the long run, this shift away from password dependency helps organizations build a future-ready identity management strategy that is both secure and user-friendly.

5.6 Security Hardening and Real-Time Monitoring

The system was developed with strong attention to security, applying multiple hardening techniques to defend against common cyber threats. One important safeguard is brute-force protection, which blocks repeated failed login attempts to prevent attackers from guessing passwords. Another is replay attack prevention, which ensures that authentication tokens cannot be stolen and reused by malicious actors. These measures make the authentication process more reliable and resilient against attacks.

Another key feature is back-channel logout, which strengthens session management. When a user logs out from one application, the system automatically ends their session across all other connected services. This prevents forgotten or inactive sessions from being exploited and ensures consistent security across the environment.

By combining these protections, the framework creates a layered defense model that reduces the risk of unauthorized access while maintaining user convenience. From a student perspective, the key takeaway is that even simple but well-integrated measures—like blocking brute-force attempts, preventing token reuse, and enforcing cross-application logout—can significantly raise the overall security of a multi-tenant identity system.

5.7 Scalability and Performance

One of the main objectives of this project was to ensure that the system could scale effectively in large and dynamic environments. To achieve this, the framework was designed with horizontal scalability, where Keycloak instances can run in a cluster and be managed behind NGINX for load balancing and traffic distribution. This setup allows the system to handle increased demand by simply adding more servers, ensuring that no single component becomes a bottleneck.

The architecture also supports high availability, meaning that if one server fails, other nodes in the cluster continue to provide authentication services without interruption. This makes the framework reliable for enterprise-level applications where downtime is unacceptable.

During testing, the system was connected with multiple service providers (SPs) and identity providers (IdPs). The results showed that it could process a large number of authentication requests at the same time while maintaining stable performance. Even under heavy load, there was no significant drop in response time, proving that the system is both efficient and resilient.

5.8 Real-World Application and Integration

The practical utility of the system was demonstrated by integrating multiple real-world applications as Service Providers (SPs). This showed how the framework can be applied in diverse organizational settings. The key points are:

- **Student Management System (app1.project.com):**
 - Integrated using OIDC.
 - Provided secure access to student records, enrollment, and academic activities.
 - Ensured role-based separation for students, teachers, and administrators.
- **Cyber Range Scoring System (app2.project.com):**
 - Integrated using SAML.
 - Enabled participants to authenticate once and access secure virtual environments for cybersecurity challenges.
 - Authentication and role mapping ensured that trainers, students, and evaluators had appropriate access levels.
- **Training Platform (app3.project.com):**
 - Integrated using OIDC.
 - Delivered training content, assessments, and course materials with seamless login.
 - Supported multiple roles such as trainees, trainers, and coordinators, with tenant-aware access policies.

- **Seamless Single Sign-On (SSO):**

- Once authenticated, users gained access across all integrated systems without re-entering credentials.
- Reduced login fatigue and improved user productivity.

- **Validation of Framework:**

- Proved that the architecture works in real-world scenarios.
- Showcased adaptability to different protocols (SAML and OIDC) and diverse applications.
- Demonstrated potential use in industries such as education, corporate training, and cybersecurity.

This integration validates that the proposed SSO framework is not only technically sound but also ready for enterprise adoption, where different applications and identity providers must work together securely and efficiently.

5.9 Compliance and Observability

The system was designed with compliance in mind, making sure it can meet the requirements of widely recognized standards such as ISO 27001, GDPR, and other security frameworks. To achieve this, the framework includes detailed logging, monitoring, and auditing features that provide a complete record of activities across the platform. Every authentication attempt, user action, and administrative change is automatically captured, creating a transparent audit trail that can be reviewed when needed.

This level of observability is important because it allows organizations to demonstrate accountability during audits or security reviews. For example, if an unusual login attempt occurs or a sensitive policy change is made, administrators can trace exactly what happened, who performed the action, and when it took place. Such visibility helps in both detecting threats quickly and proving compliance to regulators.

Another key benefit is incident response readiness. Since all events are logged in detail, security teams can analyze patterns, identify anomalies, and take corrective action before issues escalate. This proactive approach not only protects sensitive data but also builds trust with stakeholders who expect strong data governance.

An important insight derived from this work is that compliance should not be viewed solely as adherence to regulatory requirements; it also encompasses the principles of transparency, accountability, and trust. By embedding observability within the framework, the system extends beyond technical security measures to address broader organizational imperatives, thereby ensuring that it remains compliant, auditable, and dependable in multi-tenant environments.

5.10 Practical Contributions and Impact

This research makes several contributions to the field of IAM and federated authentication:

- **Academic Contribution:** The project provides a comprehensive framework for secure, scalable, and federated SSO solutions in multi-tenant environments, offering valuable insights into the challenges and best practices for implementing IAM in complex cloud infrastructures.
- **Real-World Application:** By developing a working prototype using open-source tools (Keycloak, PostgreSQL, NGINX), this project offers a deployable solution that organizations can adopt without incurring high costs. The system is extensible, allowing for easy integration with future identity providers and service providers.
- **Security Best Practices:** The implementation of passwordless authentication, real-time monitoring, and robust security features ensures that the system not only meets current security standards but also provides a foundation for future advancements in IAM and federated identity systems.

In conclusion, the project successfully developed and implemented a scalable, secure, and flexible SSO framework capable of meeting the demands of modern, federated, and multi-tenant cloud environments. By addressing key challenges such as cross-domain federation, tenant isolation, and passwordless authentication, the framework offers a comprehensive solution that enhances security, simplifies user experience, and ensures compliance with regulatory standards. The implementation of this system can significantly improve the way organizations manage identity and access across diverse applications and platforms, making it a valuable contribution to the field of identity and access management.[11]

Bibliography

- [1] AHN, G. J. Analyzing multi-tenant security frameworks in federated cloud environments for scalable enterprise applications. *ResearchGate Preprint* (2025).
- [2] ALAMU, R., AND WALKER, M. Federated identity and access in space-focused multi-tenant clouds. *ResearchGate Preprint* (2025).
- [3] BAKKER, E., ET AL. Cross-domain authentication and authorization in multi-cloud environments. *Journal of Cloud Infrastructure* 15, 2 (2020), 43–58.
- [4] BAZAZ, T., AND KHALIQUE, A. A review on single sign on enabling technologies and protocols. *International Journal of Computer Applications* 151, 11 (2016), 18–25.
- [5] DEOCHAKE, S., AND CHANNAPATTAN, V. Identity and access management framework for multi-tenant resources in hybrid cloud computing. *arXiv Preprint* (2025).
- [6] DESHMUKH, S., AND ADEKUNLE, O. Federated identity and single sign-on in iot ecosystems. *IoT Security Journal* 3, 2 (2021), 56–66.
- [7] GUL, M., AND TURAN, B. Federated identity management: A review of protocols and security features. *International Journal of Computer Applications* 179, 8 (2019), 12–24.
- [8] GUSTAFSON, A., AND LEE, Y. Access control and identity federation in multi-tenant cloud environments. *Cloud Computing & Security* 12, 1 (2017), 23–35.

- [9] GÜLTEN, A. Federated identity and access management: A review of approaches and protocols. *Journal of Cloud Computing* 9, 1 (2022), 101–112.
- [10] HARRIS, J., AND MALEK, F. Multi-factor authentication and passwordless login in federated iam systems. *International Journal of Information Security* 19, 4 (2020), 233–244.
- [11] HEVNER, A. R., MARCH, S. T., PARK, J., AND RAM, S. Design science in information systems research. *MIS Quarterly* 28, 1 (2004), 75–105.
- [12] INDU, S., AND ANAND, A. Modular iam in service-oriented architectures (soa) and cloud platforms. *Journal of Cloud Computing* 7, 3 (2015), 54–65.
- [13] KOHL, J., AND ZHANG, H. Evaluating role-based access control (rbac) and attribute-based access control (abac) in iam systems. *Journal of Network and Systems Management* 28, 3 (2019), 89–103.
- [14] LEANDRO, M. A. P., NASCIMENTO, T. J., DOS SANTOS, D. R., WESTPHALL, C. M., AND WESTPHALL, C. B. Multi-tenancy authorization system with federated identity for cloud-based environments using shibboleth. In *Proceedings of the Eleventh International Conference on Networks (ICN)* (2012).
- [15] MANSON, K., AND COLLE, R. Evaluating the effectiveness of single sign-on in federated environments. *Journal of Cloud Computing* 8, 2 (2019), 65–75.
- [16] POZO HUERTAS, E. Cloud-native iam in federated environments: Challenges and best practices. *Journal of Cloud Security and Compliance* 5, 1 (2022), 1–12.
- [17] SUBASHINI, S., AND KAVITHA, V. A survey on security issues in service delivery models of cloud computing. *Journal of Network and Computer Applications* 34, 1 (2011), 1–11.
- [18] SUN, D., DENG, H., AND MAO, R. An improved single sign-on model: Design and implementation. *International Journal of Computer Applications* 173, 3 (2021), 34–43.

- [19] TANG, M., AND SANDHU, R. A cross-tenant trust model for cloud-based identity and access management. *Journal of Cloud Computing* 10, 4 (2013), 85–92.
- [20] ZHANG, Y., ET AL. Federated identity and access management: Security challenges and protocols. *International Journal of Cloud Computing and Services Science* 8, 4 (2020), 123–134.
- [21] ZHANG, Y., AND HUO, L. Federated identity management for cloud computing: A survey. *International Journal of Cloud Computing and Services Science* 6, 4 (2017), 233–248.